PostQuantumEVM: A Native Lattice-Based Transaction Type and Proof-of-Authority Consensus for Ethereum Execution Clients

Manuel M. Delgado-Gambino López

Abstract

Ethereum's transaction layer relies on ECDSA over secp256k1, an elliptic-curve scheme that Shor's algorithm reduces to polynomial time on a cryptographically relevant quantum computer (CRQC). With expert estimates placing CRQC availability within a 10–15 year horizon and a full chain migration requiring 6–13 years, the transition window is already open under the *harvest-now-decrypt-later* threat model. This paper presents **PostQuantumEVM**, a complete implementation of a quantum-resistant Ethereum execution layer that introduces (i) a native EIP-2718 transaction type `0x50` carrying an ML-DSA-65 (FIPS-204) signature and public key; (ii) a new EVM opcode `PQHASH` (`0x21`) exposing SHAKE-256 to smart contracts; (iii) a verification precompile at address `0x0100`; (iv) a Proof-of-Authority consensus engine sealing blocks with ML-DSA-65; and (v) a complete post-quantum wallet and a multi-node end-to-end test suite. The implementation modifies the *reth* client and is validated by 34 unit tests across five PQ crates, 27 end-to-end tests across single- and multi-node configurations, and quantum simulations of Shor's algorithm (ECDLP) and Grover's algorithm (hash preimage) on a reduced model. Empirical benchmarks on a Ryzen 5 7600X show ML-DSA-65 verification is approximately 14% faster than the equivalent pure-Rust ECDSA implementation (42 µs vs. 49.2 µs), at the cost of a $\sim$46$\times$ growth in transaction size (115 B vs. 5,334 B) and a $\sim$79% drop in best-case throughput on a 12 s slot. We provide an open EIP draft formalizing the type and discuss the path to a post-quantum proof-of-stake consensus, the principal obstacle of which remains the absence of a standardized lattice-based aggregable signature scheme.

Index Terms

Post-quantum cryptography, blockchain, Ethereum, ML-DSA, lattice-based signatures, FIPS-204, EIP-2718, quantum threat, Shor's algorithm, Grover's algorithm.

I. Introduction

THE security of Ethereum's transaction layer [1], [2] ultimately reduces to two computational problems: the elliptic-curve discrete logarithm problem (ECDLP) over secp256k1, which authenticates every externally-originated transaction through ECDSA, and the preimage resistance of Keccak-256, which protects state and transaction integrity through the Merkle Patricia Trie. Both assumptions are concretely undermined by quantum algorithms: Shor's algorithm [3] solves ECDLP in polynomial time on a CRQC, while Grover's algorithm [4] halves the effective preimage security of any hash function.

The 2024 publication of the first three NIST post-quantum standards (ML-KEM in FIPS-203 [5], ML-DSA in FIPS-204 [6], and SLH-DSA in FIPS-205 [7]) closes the algorithmic gap and enables concrete protocol-level migrations. The remaining engineering question is how to integrate these primitives into a live, contract-rich blockchain without breaking application compatibility or imposing a hard cut-over.

Existing work in this space has so far stopped short of a functional client-level implementation: surveys characterize the threat surface and enumerate algorithm choices [8]–[10]; recent Ethereum Improvement Proposals (EIP-7932 [11], EIP-8051 [12]) sketch the specification of a secondary signature algorithm and an ML-DSA verification precompile but do not include a client implementation, a wallet, or multi-node validation; Hyperledger Fabric prototypes [13] demonstrate ML-DSA at the application layer in a permissioned setting that lacks the dynamics of a public EVM chain.

Manuel M. Delgado-Gambino López is with the School of Engineering and Information Technologies, Universidad Intercontinental de la Empresa (UIE), Madrid, Spain (e-mail: manuelmateodgl02@gmail.com).

Manuscript prepared as the technical report companion to the undergraduate final project "Analysis and Mitigation of Quantum Vulnerabilities in Blockchain Networks (EVM): a Lattice-Based Cryptography Approach," Bachelor's Degree in Intelligent Systems Engineering, May 2026.

Contributions

This paper makes the following contributions.

1) We define and implement a new EIP-2718 typed transaction `0x50` that carries an ML-DSA-65 signature (3,309 B) and public key (1,952 B), with explicit domain separation in the signing hash to prevent cross-type replay.
2) We extend the EVM with two PQ-native primitives: an opcode `PQHASH` (`0x21`) computing SHAKE-256 with the same gas model as `KECCAK256`, and a verification precompile at `0x0100` that accepts (h, σ, pk) tuples and returns an ABI-encoded boolean.
3) We replace ECDSA-vulnerable precompiles (`ecrecover`, BN254 pairings, KZG, BLS12-381) with revert stubs and design a Proof-of-Authority consensus engine that seals blocks with ML-DSA-65 in round-robin rotation.
4) We deliver a complete post-quantum wallet (CLI and TUI), a multi-node Docker Compose / Kubernetes deployment, and a 15-scenario end-to-end test suite covering chain identity, consensus, fee model, PQ transactions, smart contracts, and multi-node consistency.
5) We validate the threat model with quantum simulations: Shor's algorithm against a reduced toy curve $y^2 = x^3 + 7 \pmod{17}$ with generator $(6, 11)$ and order 18 (recovering the private key for all 17 non-trivial scalars), and Grover's algorithm against a 4-bit reduced Keccak (verifying the $\sqrt{N}$ speedup).
6) We provide empirical performance numbers on a Ryzen 5 7600X, with a formal EIP draft (`eip-pq-tx.md`) that documents specification, rationale, backwards compatibility, security considerations, and reference implementation.

To the best of the author's knowledge, this is the first end-to-end functional implementation of a native post-quantum transaction type in a modified Ethereum execution client with multi-node consensus validation.

The remainder of the paper is organized as follows. Section II reviews ECDSA, Shor's and Grover's algorithms, lattice-based cryptography, and the relevant Ethereum substrate. Section III situates the work against the state of the art. Section IV describes the system design. Section V discusses implementation details and deployment. Section VI reports on the quantum simulations. Section VII presents the experimental evaluation. Section VIII provides a security analysis. Section IX discusses limitations and future work. Section X concludes.

II. Background and Threat Model

A. ECDSA and the Quantum Threat to ECDLP

Ethereum uses ECDSA over secp256k1, the Koblitz curve $y^2 = x^3 + 7$ over $\mathbb{F}_p$ with $p = 2^{256} - 2^{32} - 977$. The security of every signature reduces to the intractability of ECDLP: given $Q = k \cdot G$ for a public generator G, recover k. Classical algorithms (Pollard ρ, baby-step giant-step) achieve $\Theta(\sqrt{n}) \approx 2^{128}$ for the group of order $n \approx 2^{256}$.

Shor's algorithm reduces ECDLP to a hidden subgroup problem over $\mathbb{Z}_n \times \mathbb{Z}_n$ and recovers k in $O(\log^3 n)$ quantum operations. Recent resource estimates [14] place a fault-tolerant attack on secp256k1 at ∼2,330 logical qubits, translating to millions of physical qubits under realistic error-correction overheads. Independent surveys of the quantum hardware community [15] place the probability of a CRQC within 15 years above 50%.

Two structural facts make the Ethereum exposure particularly acute. First, every account that has ever signed a transaction has its public key on-chain (recoverable from (v, r, s) via `ecrecover`). Second, the *harvest-now-decrypt-later* model means that signatures produced today are at risk retroactively: an adversary can record all current transactions and recover the corresponding private keys once a CRQC is available, without needing real-time access.

B. Grover's Algorithm and Hash Functions

For an unstructured search over $N = 2^n$ inputs with M marked elements, Grover's algorithm requires $O(\sqrt{N/M})$ oracle queries [4]. Applied to preimage search, this halves the effective security: Keccak-256, classically offering 2^{256} preimage resistance, drops to 2^{128} quantum operations—still infeasible. For collision search, the BHT algorithm [16] achieves $\Theta(2^{n/3})$ quantum queries with $\Theta(2^{n/3})$ memory, yielding 2^{85} for a 256-bit hash; this is the principal reason why output lengths below 256 bits are discouraged in post-quantum design.

C. Lattice-Based Cryptography and ML-DSA

A lattice $\mathcal{L}(\mathbf{B}) = \{\sum z_i \mathbf{b}_i : z_i \in \mathbb{Z}\}$ in $\mathbb{R}^n$ defines a family of computational problems—SVP (shortest vector), CVP (closest vector), SIS (short integer solution)—that admit *worst-case to average-case* reductions [17]–[19]: an algorithm solving a random instance implies one solving the worst case of approximate SVP. No known quantum algorithm achieves the asymptotic speedup over these problems that Shor delivers for factoring and discrete logarithms.

Module-Learning-With-Errors (Module-LWE), the assumption underlying ML-DSA, operates over $R_q = \mathbb{Z}_q[X]/(X^{256}+1)$ with $q = 8\,380\,417$. Public keys $(\mathbf{A}, \mathbf{t})$ encode $\mathbf{t} = \mathbf{A}\mathbf{s}_1 + \mathbf{s}_2$ with $\mathbf{s}_1, \mathbf{s}_2$ small; signing uses Fiat-Shamir with aborts on a commitment $\mathbf{w} = \mathbf{A}\mathbf{y}$, releasing $(\mathbf{z}, h, c)$ where $\mathbf{z} = \mathbf{y} + c\mathbf{s}_1$. ML-DSA-65 (NIST security level III) was standardized as FIPS-204 [6] and originates from the CRYSTALS-Dilithium proposal [20].

A critical design point for this work is that FIPS-204 specifies *two* modes for the masking seed: a *deterministic* mode where the seed is derived from (sk, m) alone, and a *hedged* mode that mixes in fresh randomness. The reference NIST implementation and the RustCrypto `ml-dsa` crate used in our implementation default to hedged. In neither mode does the uniqueness of a signature depend on a signer-controlled external nonce (as it does with the r_{nonce} of ECDSA), so the structural malleability $(r, s) \leftrightarrow (r, n-s)$ that motivated EIP-2 in Ethereum does not arise. We frame ML-DSA's malleability resistance in those terms.

The companion KEM standard ML-KEM [5], [21] is used in this work as a future-facing primitive for node-to-node channels (Section IX); it is not exposed on the transaction surface.

D. Typed Transactions and the EVM Substrate

EIP-2718 [22] defines a typed-transaction envelope of the form `TransactionType` $\|$ `TransactionPayload`, where the first byte selects the payload format. Existing types include `0x00` (legacy), `0x01` (EIP-2930 access list), `0x02` (EIP-1559 [23] dynamic fees), `0x03` (EIP-4844 blob transactions), and `0x04` (EIP-7702 code delegation). Our design assigns `0x50` to post-quantum transactions; the high nibble `5` doubles as a mnemonic for *post-quantum* and leaves the range `0x05`–`0x4f` free for future classical types.

The EVM itself is a deterministic stack machine over 256-bit words, with hash primitives (`KECCAK256`), elliptic-curve precompiles (`ecrecover`, BN254, BLS12-381, KZG), and a per-opcode gas accounting mechanism. From the standpoint of post-quantum migration, we identify the following surface: (i) `ecrecover` at `0x01` is broken by Shor; (ii) BN254 (`0x06`–`0x08`), KZG (`0x0a`), and BLS12-381 (`0x0b`–`0x13`) are likewise broken by Shor through the discrete logarithm in their respective groups; (iii) hash precompiles (`0x02` SHA-256, `0x03` RIPEMD-160, `0x09` Blake2f), `IDENTITY` (`0x04`), and modular exponentiation (`0x05`) remain safe.

III. Related Work

The literature on post-quantum blockchains splits into three categories: *analyses* that characterize the threat surface; *specifications* that propose protocol-level changes; and *implementations* that integrate post-quantum primitives into a working stack. Our work belongs to the third category and is, to the best of our knowledge, the first to do so natively in a modified Ethereum execution client.

Analyses: Fernández-Caramés and Fraga-Lamas [8] provide a Nature-published overview of quantum threats to public blockchains, classifying primitives by vulnerability. Berdik et al. [9] compare quantum and post-quantum proposals across consensus, signature, and key-exchange dimensions. These works motivate but do not implement.

Specifications: EIP-7932 [11] proposes a framework for secondary signature algorithms in Ethereum, designed for future extensibility but without committing to a concrete post-quantum scheme at the type level. EIP-8051 [12] more concretely specifies an ML-DSA verification precompile, leaving the signature itself outside the transaction envelope (i.e., to be carried as calldata to a contract). Our design differs in that ML-DSA-65 is the *native* signature of the transaction, sealed within the EIP-2718 envelope and verified by the client as part of admission to the mempool.

Implementations: Kiktenko et al. [10] demonstrate a quantum-secured blockchain using QKD plus hash-based signatures in a custom stack. Lee et al. [13] prototype ML-DSA in Hyperledger Fabric. Both target permissioned settings and do not address the open-set validator scaling or the dynamics of a public EVM chain. Our prototype is, by contrast, an actual fork of *reth* [24] with native transaction type, consensus engine, and multi-node operation.

Post-quantum aggregation: A separate body of work investigates aggregable lattice-based signatures, motivated by the same scaling concern that BLS aggregation solves in the classical setting. Squirrel [25] and its successor Chipmunk [26] achieve logarithmic-size aggregate signatures under SIS, at the cost of a synchronized setup. STARK-based aggregation is a complementary approach with attractive asymptotic size at the cost of non-trivial prover time. None has reached the standardization maturity required to replace BLS12-381 in a $\sim 10^6$-validator proof-of-stake consensus; we adopt PoA as a pragmatic compromise (Section IV-F) and discuss the path forward in Section IX.

IV. System Design

The system extends the Ethereum protocol along four axes: transaction format, EVM instruction set, consensus, and client implementation. We describe each in turn.

A. Transaction Type `0x50`

A post-quantum transaction is encoded as

$$\texttt{0x50} \,\|\, \mathrm{RLP}([\texttt{chain\_id}, \texttt{nonce}, \texttt{gas\_price}, \texttt{gas\_limit}, \texttt{to}, \texttt{value}, \texttt{input}, \sigma, pk]),$$

with σ a 3,309-byte ML-DSA-65 signature and pk a 1,952-byte ML-DSA-65 public key. The signing hash is computed as

$$h_{\text{sign}} = \text{SHAKE-256}\big(\,\texttt{0x50} \,\|\, F(T)\,,\, 32\big), \tag{1}$$

where $F(T)$ is a big-endian concatenation of the typed transaction fields and the leading `0x50` byte provides explicit domain separation across EIP-2718 types. The transaction identifier is

$$h_{\text{tx}} = \text{SHAKE-256}(\texttt{0x50} \,\|\, h_{\text{sign}} \,\|\, \sigma \,\|\, pk\,,\, 32). \tag{2}$$

Two design points deserve discussion. First, ML-DSA-65 does not support public-key recovery from (m, σ) in the way ECDSA does; the public key must be transmitted explicitly with every transaction, which dominates the size cost. Second, the type-`0x50` envelope intentionally uses a flat `gas_price` (legacy fee model) rather than the EIP-1559 triple `(max_fee_per_gas, max_priority_fee_per_gas, base_fee)`; this minimizes the signed field set and is independent of whether the underlying client computes `baseFeePerGas` at the block level (which it does, since our prototype inherits reth's post-Merge fee logic). Extending `0x50` to carry the dynamic-fee fields is straightforward and is listed as future work.

B. Address Derivation

Post-quantum addresses derive from the public key via SHAKE-256:

$$\texttt{address} = \text{SHAKE-256}(pk, 32)[12{:}32]. \tag{3}$$

Using SHAKE-256 (rather than Keccak-256, which Ethereum uses for classical addresses) provides cryptographic separation between classical and PQ address spaces: even if a malicious party constructs an ML-DSA-65 public key whose Keccak-256 hash collides with an existing classical address, the resulting PQ address would not collide. The two address namespaces remain in the same 20-byte format, so existing smart contracts that operate on addresses (transfers, mappings, event topics) work without modification.

C. Opcode `PQHASH (0x21)`

We allocate the next available opcode slot adjacent to `KECCAK256` (`0x20`) to expose SHAKE-256 to contract code. `PQHASH` takes `(offset, length)` from the stack, expands memory if needed, and pushes the 32-byte SHAKE-256 output. Gas cost mirrors `KECCAK256`: 30 base plus 6 per 32-byte word. The choice of opcode over precompile is deliberate: a smart contract calling a precompile for the same operation would pay an additional $G_{\text{call}} = 700$ gas for the `STATICCALL` itself, plus 100–2,600 gas for the EIP-2929 access list (warm/cold), plus 3 gas per word of *calldata* marshalling and *returndata* copy. For a 32-byte input, this overhead is roughly $\sim$800 to $\sim$3,300 gas above what `PQHASH` charges; the opcode eliminates it entirely.

D. ML-DSA-65 Verification Precompile (0x0100)

Contract-level verification of ML-DSA-65 signatures is exposed at the precompile address 0x0100. The input is the 5,293-byte concatenation $h \,\|\, \sigma \,\|\, pk$ (32 + 3,309 + 1,952). The output is uint256(1) on valid signature, uint256(0) otherwise, ABI-encoded as 32 bytes.

The gas cost is calibrated from measured latency:

$$\text{gas}_{\text{theory}} = \lceil 42\,\mu\text{s} \cdot 61\,\text{gas}/\mu\text{s} \cdot 1.35 \rceil = 3{,}459. \tag{4}$$

The production constant is rounded down to the nearest multiple of 50, $\text{gas}_{\text{prod}} = 3{,}450$, aligning with the cost convention of existing precompiles (ecrecover: 3,000; sha256base: 60; ripemd160base: 600). The 1.35 safety factor accommodates platform variability and conservative side-channel margin.

E. Disabled Precompiles

The PoA fork disables thirteen Shor-vulnerable precompiles by replacing their implementation with a revert stub: ecrecover (0x01), BN254 add/mul/pairing (0x06–0x08), KZG point evaluation (0x0a), and the nine BLS12-381 operations (0x0b–0x13). The five quantum-safe precompiles—SHA-256 (0x02), RIPEMD-160 (0x03), IDENTITY (0x04), modular exponentiation (0x05), Blake2f (0x09)—are preserved. In a real-world deployment, the transition strategy would keep ecrecover active during a grace window to verify historical signatures; the prototype here is a forward-looking proof of concept and disables it immediately to surface any contract incompatibilities.

F. Proof-of-Authority Consensus with ML-DSA-65 Seals

A natural reflex is to retain proof-of-stake and merely substitute the validator signature algorithm. This is not yet practical: PoS on Ethereum relies on BLS12-381 aggregation to combine the attestations of $\sim 10^6$ validators into a constant-size signature, and no standardized post-quantum scheme offers comparable aggregation. Without aggregation, each block would have to carry one ML-DSA-65 signature per attesting validator—roughly $10^6 \cdot 3{,}309 \approx 3.3$ GB per block, several orders of magnitude beyond viable.

We therefore adopt Proof-of-Authority for the prototype, with a small, known validator set. Each block header carries an ML-DSA-65 seal in its extra_data field:

1) The block proposer is determined deterministically: $\text{proposer}(h) = V[h \bmod |V|]$, where V is the ordered validator set and h the block height.
2) The seal hash is $h_{\text{seal}} = \text{SHAKE-256}(\text{RLP}(\text{header with empty extra\_data}))$, with extra_data cleared to break the circularity between signed data and signature.
3) The proposer signs h_{seal} with its ML-DSA-65 private key, producing a 3,309-byte seal, and writes it into extra_data.
4) Validating peers recompute h_{seal}, look up the expected proposer for that height, and verify the seal against the proposer's public key.

The choice of PoA is not a claim that proof-of-stake is unsalvageable in a post-quantum setting; it is a pragmatic deferral until lattice-based aggregable signatures reach standardization (see Section IX).

V. Implementation

The implementation comprises four Cargo workspaces totalling roughly 18,000 lines of Rust and Python.

A. Cryptographic Library: ml-lattice-rs

A thin workspace with two crates: dilithium (ML-DSA, FIPS-204) and kyber (ML-KEM, FIPS-203). Both wrap the corresponding RustCrypto implementations (ml-dsa 0.1.0-rc.8 and ml-kem 0.2) behind a uniform interface across the three security levels. The signing entry point uses the default hedged mode of ml_dsa::sign; the deterministic mode is exposed as sign_deterministic for callers that require byte-for-byte reproducibility. SHAKE-256 is imported directly from the sha3 crate by both pq-reth and pq-wallet; it is not re-exported by ml-lattice-rs to avoid coupling.

Table I
POST-QUANTUM CRATES IN PQ-RETH AND THEIR TEST COVERAGE

Crate	Tests	Responsibility
reth-pq-primitives	11	Transaction 0x50, RLP, sender derivation
reth-pq-poa	9	PoA consensus, ML-DSA-65 seal, validator rotation
reth-pq-evm	6	EVM extension, opcode PQHASH, precompile table
reth-pq-precompile	5	ML-DSA-65 verification precompile (0x0100)
reth-pq-consensus	4	Transaction admission rules (chain id, gas, signature)
Total	**34**	All passing under cargo test -release (rustc 1.95.0)

B. Modified Reth Client: pq-reth

The client fork organizes the post-quantum additions into five focused crates (Table I). Each is consumed at the corresponding extension point of upstream reth via the trait system, minimizing intrusion on the base codebase.

C. Post-Quantum Wallet: pq-wallet

The wallet workspace ships five crates: a pq-wallet-core library (key generation, AES-256-GCM encrypted keystore with Argon2id KDF, signer, transaction builder, JSON-RPC client, error types); a pq-wallet-cli binary with nine subcommands (new, address, balance, send, deploy, receipt, sign, call, export-seed); a pq-wallet-tui interactive terminal interface built on ratatui 0.29 and crossterm 0.28, with four tabs (wallet state, transactions, blocks, network); a pq-e2e-test runner for the end-to-end suite; and a pq-chain-seeder that primes a fresh chain with sample state for demonstrations.

D. Quantum Simulation Service: qiskit-api

A FastAPI service (Python 3.13, Qiskit 2.3) exposing the Shor and Grover simulations as REST endpoints (POST /shor/ecdlp, POST /grover/keccak, plus auxiliary GET endpoints for curve and hash tables). The Shor implementation uses an exact 2D modular DFT to avoid the spectral leakage that occurs when the group order is not a power of two; the Grover circuit is built natively from Qiskit gates and executed on the Aer simulator.

E. Deployment

A Docker Compose definition deploys a three-validator PoA cluster plus the simulation service on an isolated bridge network, with explicit trusted peers (P2P discovery disabled to make node identity deterministic). A parallel set of Kubernetes manifests deploys the same topology as a StatefulSet with per-pod config selection via an initContainer that reads the ordinal of the pod from its hostname. The genesis file uses Chain ID = 20561, Terminal Total Difficulty = 0 (to activate the post-Merge engine API path on which the PoA scheduler attaches), and all hard forks active from block zero.

VI. QUANTUM SIMULATIONS

The simulations validate experimentally that the threat model is concrete and reproducible, even on a small instance solvable on a classical Qiskit Aer back-end.

A. Shor's Algorithm on a Toy Curve

We use the reduced curve $E : y^2 = x^3 + 7 \pmod{17}$, preserving the Koblitz form of secp256k1 ($a = 0$, $b = 7$) but reducing the prime to 17. Direct enumeration verifies $|E(\mathbb{F}_{17})| = 18$ (17 affine points plus $\mathcal{O}$). The Hasse bound is trivially satisfied: $|p + 1 - |E|| = |17 + 1 - 18| = 0 \leq 2\sqrt{17} \approx 8.25$.

Generator selection. The group $E(\mathbb{F}_{17})$ is cyclic of order 18; subgroup orders are $\{1, 2, 3, 6, 9, 18\}$. Exactly six affine points have order 18: $\{(6, 6), (6, 11), (10, 2), (10, 15), (15, 4), (15, 13)\}$. Points $(1, 5)$, $(1, 12)$, $(2, 7)$, $(2, 10)$, $(12, 1)$, $(12, 16)$ have order 9 and generate only a proper subgroup; selecting one of them would silently restrict the

Table II
CRYPTOGRAPHIC OPERATION LATENCY: ECDSA (K256) VS. ML-DSA-65

Operation	ECDSA (µs)	ML-DSA-65 (µs)	Ratio
Key generation	35.1	208.5	5.94× slower
Sign	41.4	342.8	8.28× slower
Verify	**49.2**	**42.0**	**0.85× (14% faster)**

key space to $\mathbb{Z}_9$. We choose $G = (6, 11)$, verified to have order 18 by enumeration ($k \cdot G \neq \mathcal{O}$ for $k = 1, \ldots, 17$; $18 \cdot G = \mathcal{O}$).

Algorithm. For an unknown secret $k \in \{1, \ldots, 17\}$ and public $Q = k \cdot G$, the simulation prepares the coset state

$$|\psi\rangle = \frac{1}{\sqrt{18}} \sum_{b=0}^{17} \big|(p_0 - bk) \bmod 18\big\rangle_a \, |b\rangle_b, \tag{5}$$

where p_0 is the index of the point measured on the output register. A 2D modular DFT of size 18×18 is then applied; measuring (s, t) from the resulting distribution and discarding pairs with $\gcd(s, 18) \neq 1$, the secret is recovered as $k = s^{-1} \cdot t \bmod 18$. With 4,096 shots, all 17 non-trivial secrets are recovered correctly. A worked example with $k = 7$ ($Q = (15, 13)$) and a measured pair $(s, t) = (5, 17)$ yields $s^{-1} = 11$, $k = 11 \cdot 17 \bmod 18 = 7$ as expected.

B. Grover's Algorithm on a Reduced Keccak

A toy Keccak operates on an 8-bit state with a 4-bit rate, 4-bit capacity, and two rounds, yielding a 4-bit hash—a search space of $N = 16$. For a single-preimage target ($M = 1$), the optimal iteration count is $k_{\text{opt}} = \lfloor (\pi/4)\sqrt{N} \rfloor = 3$, and the algorithm finds the correct preimage with probability >95% over 1,024 shots. For multi-preimage targets ($M > 1$), the iteration count adapts as $\lfloor (\pi/4)\sqrt{N/M} \rfloor$ and the per-shot success probability rises accordingly. Extrapolated to Keccak-256 ($N = 2^{256}$), Grover requires $\sim 2^{128}$ oracle queries [27], the level of post-quantum security we accept as adequate.

VII. EVALUATION

All benchmarks were run on an AMD Ryzen 5 7600X (6 cores, 12 threads, AVX2 and AVX-512 available; `schedutil` performance governor without fixed frequency), 32 GB DDR5-5600, Linux 7.0.9 `cachyos`, `rustc 1.95.0` in `release` mode with `lto=true`, `codegen-units=1`, `target-cpu=native`. ML-DSA-65 is exercised through `ml-dsa 0.1.0-rc.8` (RustCrypto, AVX2 path when available); ECDSA over secp256k1 through `k256 0.13` (RustCrypto, pure-Rust). Numbers are medians over 100 Criterion.rs samples with MAD-based outlier rejection.

A. Cryptographic Latency

Table II compares per-operation latency.

The salient result is that *verification* of ML-DSA-65 is approximately 14% faster than ECDSA verification under a like-for-like pure-Rust comparison. This is the operation that every node performs on every transaction of every block, so the result generalizes favourably to the steady-state cost of running a node. The 8.28× penalty on signing is borne by the user-side wallet, not by the network. The 5.94× penalty on key generation is paid once per account.

A caveat: the comparison would shift if ECDSA were measured against `libsecp256k1` (the C implementation specifically optimized for Bitcoin), which is roughly 3–5× faster than `k256`. The qualitative conclusion (ML-DSA-65 verification is *competitive* with ECDSA, not categorically slower) holds, but the specific 14% margin is library-dependent.

Table III
HASH FUNCTION LATENCY: KECCAK-256 VS. SHAKE-256 (NANOSECONDS)

Input	Keccak-256	SHAKE-256	Ratio
32 B	300	545	1.82×
64 B	304	560	1.84×
128 B	301	573	1.90×
256 B	555	802	1.45×
512 B	1,089	1,322	1.21×
1 KB	2,054	2,329	1.13×
4 KB	8,130	8,100	1.00×
8 KB	16,229	15,608	0.96×

Table IV
TRANSACTION SIZE: CLASSICAL (ECDSA) VS. POST-QUANTUM (ML-DSA-65)

Component	Classical (B)	PQ (B)
RLP header, nonce, gas, value	30	53
Recipient address	20	20
Signature	65 (v, r, s)	3,309
Public key	(recoverable)	1,952
Total (component sum)	**115**	**5,334**
Total (on-the-wire)†	∼110	∼5,299
Ratio	$\sim \mathbf{46}\times$	

† RLP encodes integers < 128 in a single byte, so the encoded transaction is slightly smaller than the sum of maximum component sizes.

B. Hash Functions

Table III compares Keccak-256 (Ethereum's native hash) and SHAKE-256 (used by ML-DSA, address derivation, and `PQHASH`) across input sizes.

The two functions share the same Keccak-$f[1600]$ permutation; they differ only in padding and initialization. SHAKE-256 carries a ∼250 ns constant overhead on small inputs, which dominates below 256 B. As inputs grow, the per-byte cost converges and the two functions become indistinguishable around 4 KB. The principal use case in this work—address derivation from a 1,952-byte public key—falls in the regime where the overhead is ∼10–15%, negligible relative to the cost of the ML-DSA verification itself.

C. Transaction Size

Table IV decomposes the per-component size of a minimal ETH transfer (empty `input`) under both schemes.

The ∼46× size increase is the dominant cost of the migration. Of the ∼5,220 added bytes, 63% are the signature itself and 37% the public key; a future on-chain public-key registry could amortize the latter across transactions, reducing the size penalty by approximately 37% for recurring senders.

D. Throughput and Gas

Table V converts size and gas costs into achievable throughput.

The ∼79% throughput loss on a mainnet-equivalent configuration is dominated by calldata gas, not by cryptographic computation. On-chain public-key registration would reduce this loss to roughly 50%.

E. End-to-End Validation

The end-to-end suite executes 15 scenarios in six phases: chain identity (3), PoA consensus (2), EIP-1559 fee model (2), PQ transactions (3), smart contracts (2), and multi-node consistency (3). In the single-node configuration, 12/12 tests pass in 26.63 s. In the three-node configuration, an additional 3/3 multi-node tests pass in 29.98 s with a measured drift bounded by 2 blocks across nodes (consistent with a 3 s slot and observed P2P propagation latency). Block-state consistency across the three nodes is verified by RPC sampling of account balances after each phase. All 34 unit tests across the five PQ crates also pass.

Table V
GAS AND THROUGHPUT IMPACT

Metric	Classical	PQ
Gas per transfer (incl. calldata)	21,976	42,892
Gas for crypto verification	3,000 (`ecrecover`)	3,450 (ML-DSA)
Gas for calldata (sig + pk)	$65 \times 16 = 1{,}040$	$5{,}261 \times 16 = 84{,}176$
Total minimum per transaction	21,976	105,176
Peak TPS, mainnet (30M gas, 12 s slot)[a]	$\sim$114	$\sim$24
Peak TPS, prototype (30M gas, 3 s slot)[b]	$\sim$455	$\sim$95
Throughput loss (mainnet)	—	$\sim$79%

[a] Classical: $\lfloor 30\text{M}/21{,}976 \rfloor / 12 \approx 114$ TPS. PQ: $\lfloor 30\text{M}/105{,}176 \rfloor / 12 \approx 24$ TPS.
[b] Same per-block capacity; the prototype uses a 3 s slot (cf. Section V).

VIII. SECURITY ANALYSIS

We consider four adversary models.

A. Replay

A type-`0x50` transaction binds `chain_id` and `nonce` into h_{sign} (Eq. (1)) and prefixes the byte `0x50`, providing domain separation. Cross-chain replay fails because h_{sign} changes with `chain_id`; replay against the same chain fails the mempool's nonce check (`NonceTooLow`); cross-type replay fails because a payload signed for `0x50` carries the `0x50` byte inside its hash and is not interchangeable with a similarly-shaped `0x02` payload.

B. DoS via Transaction Inflation

The 46$\times$ size growth means an attacker can flood the mempool with proportionally more bytes for the same gas budget. Two implemented mitigations cap the damage: (i) a per-transaction size limit of 128 KiB (inherited from reth), bounding worst-case memory; (ii) the block gas limit (30M) implicitly limits per-block bytes to roughly $\lfloor 30\text{M}/105\text{K} \rfloor \approx 285$ transactions or $\sim$1.5 MB. A third mitigation—ordering the mempool by gas_price/size_bytes rather than purely by gas_price—is documented in the EIP draft as a recommended future addition.

C. Signature Malleability

ECDSA admits the structural malleability $(r, s) \leftrightarrow (r, n-s)$, which Ethereum patched with EIP-2's $s \leq n/2$ rule. ML-DSA-65 has no analogous structural malleability: as discussed in Section II-C, the uniqueness of a valid signature does not depend on a signer-controlled external nonce, and no known closed-form transformation $\sigma \to \sigma'$ produces a second valid signature for the same (m, pk). Additionally, the transaction hash (Eq. (2)) covers the full envelope including σ and pk, so a front-runner cannot replay a modified transaction under the same identifier.

D. Quantum Hashing Oracles

Keccak-256 (Merkle Patricia Trie, logs) and SHAKE-256 (address derivation, signing hash) both offer 256-bit outputs. Under Grover's algorithm, preimage resistance drops to $\Theta(2^{128})$, computationally infeasible. Collision resistance under the BHT algorithm is $\Theta(2^{n/3}) \approx 2^{85}$ with $\Theta(2^{85})$ memory, also infeasible but a reason to retain 256-bit outputs and not truncate. No known distinguisher reaches $< 2^{256}$ queries against Keccak-$f[1600]$.

IX. DISCUSSION AND LIMITATIONS

A. Limitations of the Prototype

The prototype runs on a private chain with three validators and 15 pre-funded accounts; it has not been exercised at mainnet transaction volumes or validator counts. The PoA consensus does not scale to the open-set validator population of PoS. The `0x50` type uses a flat `gas_price` rather than the EIP-1559 dynamic fee model. Each transaction carries the full public key (no on-chain registry yet). The Shor and Grover simulations operate on reduced-size models that demonstrate the principle but not the resource cost on real curves. The cryptographic

implementation has not been formally audited; the RustCrypto `ml-dsa` crate is well-tested against NIST KATs but has not, to our knowledge, undergone an independent third-party audit. Only one execution client (`reth`) is modified; a real-network migration requires the same protocol changes across `geth`, `Nethermind`, `Besu`, and `Erigon`.

B. The Path to PQ Proof-of-Stake

The principal obstacle to extending this work to PoS is the absence of a standardized post-quantum aggregable signature scheme. Three research directions are active. *Squirrel* [25] achieves logarithmic-size aggregate signatures under SIS with a synchronized setup; its successor *Chipmunk* [26] refines per-signature size and verification cost. *STARK-based aggregation* wraps a batch of ML-DSA verifications in a succinct STARK proof; the proof size is asymptotically constant in the number of signatures, at the cost of non-trivial prover time. None of these has yet reached the standardization maturity (NIST, ETSI, IETF) required to replace BLS12-381 at the scale of a $\sim 10^6$-validator consensus, but their evolution should be tracked.

C. Account Abstraction as Complementary Path

ERC-4337 [28] and EIP-7702 enable smart contracts to verify arbitrary signatures, including ML-DSA-65 via the precompile at `0x0100`. These mechanisms are not a replacement for a native type (they leave consensus-layer signatures vulnerable), but they are complementary: they offer a migration path for application-layer use cases without waiting for a coordinated hard fork, and they let existing dApps adopt PQ verification per-contract.

D. Future Work

We identify nine concrete future-work items: an on-chain public-key registry; full EIP-1559 fields in `0x50`; investigation of PQ signature aggregation schemes; ML-DSA compression standards; large-scale load testing; consensus-layer migration; cryptographic agility (a versioning scheme that supports future PQ algorithms without a hard fork); a formal EIP submission to Ethereum Magicians and All Core Devs; cross-chain bridges with native ML-DSA verification.

X. Conclusion

We have presented a complete, functional implementation of a native post-quantum transaction type for an Ethereum execution client. The implementation modifies the `reth` client, introduces a new EIP-2718 type `0x50` carrying ML-DSA-65 signatures, adds a `PQHASH` opcode and an ML-DSA verification precompile to the EVM, replaces Shor-vulnerable precompiles with revert stubs, and seals blocks with ML-DSA-65 under a Proof-of-Authority consensus engine. The system is delivered with a wallet (CLI and TUI), multi-node Docker and Kubernetes deployments, a 15-scenario end-to-end test suite, and Qiskit-based simulations of Shor's and Grover's algorithms on reduced models.

The empirical results support the central claim that the migration is technically viable today. ML-DSA-65 verification is approximately 14% faster than ECDSA under a like-for-like pure-Rust comparison; the principal cost is a $\sim 46\times$ transaction-size increase, which translates to a $\sim 79\%$ best-case throughput loss on a mainnet-equivalent 12-second slot. The hardest open problem is at the consensus layer: a post-quantum proof-of-stake requires aggregable lattice-based signatures that, while actively researched (Squirrel, Chipmunk, STARK-based aggregation), have not yet reached standardization.

The work is released as open source, with a formal EIP draft accompanying the prototype, in the hope that it serves as a concrete reference for the inevitable migration of the Ethereum transaction layer ahead of the CRQC horizon.

Acknowledgments

The author thanks the Universidad Intercontinental de la Empresa for the academic supervision of this project, and the RustCrypto maintainers for their pure-Rust implementations of ML-DSA and `k256`, which made the comparative benchmarks tractable.

REFERENCES

[1] G. Wood, "Ethereum: A secure decentralised generalised transaction ledger," Ethereum Foundation, Tech. Rep., 2014, yellow Paper, updated periodically.

[2] V. Buterin, "A next-generation smart contract and decentralized application platform," Ethereum White Paper, 2014.

[3] P. W. Shor, "Algorithms for quantum computation: Discrete logarithms and factoring," in *Proc. 35th Annu. Symp. Found. Comput. Sci. (FOCS)*. IEEE, 1994, pp. 124–134.

[4] L. K. Grover, "A fast quantum mechanical algorithm for database search," in *Proc. 28th Annu. ACM Symp. Theory Comput. (STOC)*. ACM, 1996, pp. 212–219.

[5] National Institute of Standards and Technology, "Module-lattice-based key-encapsulation mechanism standard," NIST, Federal Information Processing Standards Publication FIPS 203, Aug. 2024.

[6] ——, "Module-lattice-based digital signature standard," NIST, Federal Information Processing Standards Publication FIPS 204, Aug. 2024.

[7] ——, "Stateless hash-based digital signature standard," NIST, Federal Information Processing Standards Publication FIPS 205, Aug. 2024.

[8] T. M. Fernández-Caramés and P. Fraga-Lamas, "Quantum resistance in blockchain networks," *Scientific Reports*, vol. 13, p. 5765, 2023.

[9] D. Berdik, S. Otoum, N. Schmidt, D. Porter, and Y. Jararweh, "A survey and comparison of post-quantum and quantum blockchains," *IEEE Access*, vol. 11, pp. 116 688–116 703, 2023.

[10] E. O. Kiktenko, N. O. Pozhar, M. N. Anufriev, A. S. Trushechkin, R. R. Yunusov, Y. V. Kurochkin, A. I. Lvovsky, and A. K. Fedorov, "Quantum-secured blockchain," *Quantum Sci. Technol.*, vol. 3, no. 3, p. 035004, 2018.

[11] Ethereum Community, "EIP-7932: Secondary signature algorithms," Ethereum Improvement Proposals, 2025.

[12] ——, "EIP-8051: ML-DSA verification precompile," Ethereum Improvement Proposals, 2025.

[13] J. Lee, S. Kim, and Y. Park, "Quantum-safe blockchain in Hyperledger Fabric," *IEEE Access*, vol. 12, pp. 1–15, 2024.

[14] Y. Taguchi and A. Takayasu, "Choosing coordinate forms for solving ECDLP using Shor's algorithm," *arXiv preprint arXiv:2502.12441*, 2025.

[15] M. Mosca and M. Piani, "Quantum threat timeline report," Global Risk Institute, Tech. Rep., 2023.

[16] G. Brassard, P. Høyer, and A. Tapp, "Quantum cryptanalysis of hash and claw-free functions," in *LATIN'98: Theoretical Informatics*, ser. Lecture Notes in Computer Science, vol. 1380. Springer, 1998, pp. 163–169.

[17] M. Ajtai, "Generating hard instances of lattice problems," in *Proc. 28th Annu. ACM Symp. Theory Comput. (STOC)*. ACM, 1996, pp. 99–108.

[18] O. Regev, "On lattices, learning with errors, random linear codes, and cryptography," in *Proc. 37th Annu. ACM Symp. Theory Comput. (STOC)*. ACM, 2005, pp. 84–93.

[19] A. K. Lenstra, H. W. Lenstra, and L. Lovász, "Factoring polynomials with rational coefficients," *Math. Annalen*, vol. 261, no. 4, pp. 515–534, 1982.

[20] L. Ducas, E. Kiltz, T. Lepoint, V. Lyubashevsky, P. Schwabe, G. Seiler, and D. Stehlé, "CRYSTALS-Dilithium: A lattice-based digital signature scheme," *IACR Trans. Cryptogr. Hardw. Embed. Syst.*, vol. 2018, no. 1, pp. 238–268, 2018.

[21] J. Bos, L. Ducas, E. Kiltz, T. Lepoint, V. Lyubashevsky, J. M. Schanck, P. Schwabe, G. Seiler, and D. Stehlé, "CRYSTALS-Kyber: A CCA-secure module-lattice-based KEM," in *Proc. IEEE Eur. Symp. Secur. Privacy (EuroS&P)*. IEEE, 2018, pp. 353–367.

[22] M. Zoltu, "EIP-2718: Typed transaction envelope," Ethereum Improvement Proposals, 2020.

[23] V. Buterin, E. Conner, R. Dudley, M. Slipper, I. Norden, and A. Bakhta, "EIP-1559: Fee market change for ETH 1.0 chain," Ethereum Improvement Proposals, 2019.

[24] Paradigm, "Reth: Modular, contributor-friendly and blazing-fast Ethereum client in Rust," https://github.com/paradigmxyz/reth, 2024.

[25] N. Fleischhacker, M. Simkin, and Z. Zhang, "Squirrel: Efficient synchronized multi-signatures from lattices," in *Proc. 2022 ACM SIGSAC Conf. Comput. Commun. Secur. (CCS)*. ACM, 2022, pp. 1109–1123.

[26] ——, "Chipmunk: Better synchronized multi-signatures from lattices," in *Proc. 2023 ACM SIGSAC Conf. Comput. Commun. Secur. (CCS)*. ACM, 2023, pp. 386–400.

[27] J. Luo, Q. Ye, and X.-s. Lin, "Quantum resource analysis of Keccak/SHA-3: Classical to quantum using Qiskit modeling," *arXiv preprint arXiv:2512.14759*, 2024.

[28] V. Buterin, Y. Weiss, D. Tirosh, S. Nacson, A. Forshtat, K. Gazso, and T. Hess, "ERC-4337: Account abstraction using alt mempool," Ethereum Improvement Proposals, 2021.